

总线与 I/O 硬件

同步处理器怎样与异步设备完成可确认的协作

408 · 计算机组成原理 Topic CO-08

母问题：一次设备工作怎样从意图变成可确认的完成？

总线、控制器、轮询、中断和 DMA 不是五个孤立名词，而是一条责任链：

Request → Interface State → Arbitration → Transaction → Transfer → Completion Signal

每一步都要回答：谁拥有控制权，谁驱动共享资源，数据经过哪里，什么状态证明本步结束。

本册定位与 Owner 边界

- **Owns:** 总线角色和事务、同步/异步握手、仲裁、有效带宽、控制器寄存器、I/O 编址、polling、中断硬件入口、DMA 控制器和传输模式。
- **Uses:** CO-02 的 ISA 入口契约、CO-05/06 的内存与 Cache 接口、CO-04 的精确提交边界。
- **Stops:** 驱动策略、请求队列、block/wakeup、buffer policy、DMA mapping/unmapping 和设备调度归 OS/X-B03。

目录

1 总线是协议，不是一捆静态导线	3
2 同步、异步与分离事务	3
3 仲裁：选择共享资源的临时 Owner	3
4 带宽题先写完整事务	3
5 I/O 接口把设备变成寄存器状态机	4
6 三种控制方式：谁等待、谁搬数据、谁通知	4
7 中断硬件路径与软件交接	4
8 DMA：把搬运循环下沉到控制器	4
9 贯穿母例：DMA 读取一个块	5
10 五列概念边界	5

11 做题控制协议	5
12 全科坐标与跨册交接	6
13 考纲映射与来源处理	6

1 总线是协议，不是一捆静态导线

共享互连至少包含 initiator/master、target/slave、arbiter 和 interconnect。一次事务通常经过请求、仲裁、地址/命令、一个或多个 data beat、response 与释放。DMA 控制器获授权后也可成为 initiator；“CPU 永远是主设备”不是不变量。

地址、数据与控制信息承担不同责任：地址标识目标，数据线宽度限制每 beat 原始数据量，控制/响应表达 read/write、byte enable、ready、error、request/grant 等协议状态。地址线分时复用不会自动增加数据带宽。

2 同步、异步与分离事务

同步总线按共同 clock 采样，慢目标可插入 wait state；异步总线靠 request/acknowledge 握手建立因果关系完成关系。二者描述**定时协议**，串行/并行描述**物理传输**，不能互推快慢。

分离事务把 request 与 response 解耦，长延迟时可释放通道服务其他请求，但需要事务 ID、缓冲和匹配。是否允许多个 outstanding request 是协议属性，不由“异步”一词推出。

3 仲裁：选择共享资源的临时 Owner

仲裁只决定哪个请求者在当前时段获得发起权，不决定 OS 层的设备归属。固定优先级响应关键设备快，却可能饥饿；轮转改善公平性，却增加状态与最坏等待；链式、独立请求或分布式实现还要比较线路成本、仲裁延迟和故障范围。

机制	关键状态	优势	风险
固定优先级	pending 与 priority	关键请求低延迟	低优先级饥饿
轮转	last grant / pointer	长期公平	状态与最坏等待增加
链式查询	grant 传播位置	线路少	延迟累积、物理顺序固化
独立请求	每主设备 request/grant	并行判优快	线路和逻辑成本高

4 带宽题先写完整事务

理论峰值为

$$BW_{peak} = \frac{bytes}{beat} \times \frac{beats}{cycle} \times frequency.$$

有效带宽还要扣除仲裁、地址/响应阶段、wait、方向切换、编码与空闲。若一个事务搬运 D 字节、总耗时 T ，稳定口径是 $BW_{effective} = D/T$ 。频率、总线周期、数据周期与设备时间必须分开列。

最小反例

64 位、1 GHz 只给出每 beat 上限与时钟条件；若每次数据前有地址阶段、目标插入等待，或并非每周期都有 beat，有效带宽就不是自动的 8 GB/s。

5 I/O 接口把设备变成寄存器状态机

控制器通常暴露 data/FIFO、status、control/command，以及可选地址、计数、描述符和 interrupt mask。软件写 command 只是启动；busy、ready、done、error 才描述后续状态。

memory-mapped I/O 用普通地址空间和 load/store；isolated I/O 用独立端口空间和专用指令。差别在软件可见寻址契约，不等于设备性能。MMIO 是否可缓存、能否重排和访问宽度须服从 ISA/平台规则。

6 三种控制方式：谁等待、谁搬数据、谁通知

方式	CPU 等待	数据路径	完成发现
Polling / PIO	循环查询状态	device ↔ CPU register ↔ memory	主动读取 done
Interrupt-driven	可执行其他任务	通常仍由 CPU 指令搬运	设备异步请求入口
DMA	只配置与收尾	device ↔ memory	块完成/错误通知

中断消除的是持续 busy waiting，不自动消除 ISR 和搬运成本；DMA 下沉的是传输循环，不代表 CPU、Cache、内存和互连没有成本。

7 中断硬件路径与软件交接

device event → pending → mask/priority → precise boundary → architectural entry state → vector.

外部中断与当前指令异步；异常通常与当前指令同步相关。CPU 是否自动关中断、保存哪些最小状态、向量怎样形成均由 ISA 规定。完整通用寄存器保存、驱动服务、确认设备、唤醒进程和调度属于软件。

中断控制器的 pending、enable/mask、priority 和 in-service 状态必须分开：请求到达不等于立即接受，接受不等于 handler 已完成，handler 返回也不必自动清除设备源头。

硬件 Stop Boundary

本 Topic 的最后一个稳定动作是“按 ISA 规定形成可进入软件的体系结构状态”。“唤醒哪个任务、是否重试请求、怎样处理错误”不写成硬件动作。

8 DMA：把搬运循环下沉到控制器

CPU 先配置 source、destination、length、direction/control 或 descriptor；DMAC 获得设备和总线条件后发起 transaction，逐 beat 更新地址与计数，完成或错误时置状态并通知 CPU：

Setup → DREQ → Arbitrate → Transfer → Count/Address Update → Complete.

burst 一次授权连续传输，吞吐高但占用长；cycle stealing 每次占少量周期，在 CPU 干扰与吞吐间折中；transparent/interleaved 利用约定时隙，依赖具体时序。DREQ 是设备到 DMAC 的传输请求，不是 CPU 中断；总线 grant 也不是上下文切换。

scatter-gather 可由描述符链减少大块非连续传输的重复配置，但描述符建立、地址映射、缓存一致性和完成处理仍产生 CPU/系统成本，因此“DMA 对任意传输恒为 $O(1)$ CPU 开销”不是稳定结论。

9 贯穿母例：DMA 读取一个块

1. 软件准备设备可用的目标地址、长度和方向，并按平台规则处理映射/一致性；
2. CPU 写设备/DMAC 控制寄存器，控制器进入 busy；
3. 设备产生 DREQ，DMAC 作为潜在 initiator 请求互连；
4. arbiter 授权后，DMAC 与内存/设备完成若干 beat，更新地址和剩余计数；
5. 共享内存或总线冲突表现为等待，CPU 是否同时运行取决于其当前资源需求；
6. 计数归零或错误使控制器写 completion state，并产生可屏蔽的 interrupt request；
7. CPU 在精确边界接受中断，形成 ISA 规定的入口状态；
8. 软件确认状态、解除映射、完成请求或唤醒任务。

10 五列概念边界

概念 A	≠ 概念 B	真正区别与题面信号	混淆后果
request	≠ transaction	提出需求 vs 已获权并执行协议	漏仲裁与等待
bus grant	≠ interrupt accept	互连所有权 vs CPU 控制流入口	把暂停访存写成 ISR
DREQ	≠ IRQ	设备请求 DMAC 搬运 vs 控制器通知 CPU	错画信号方向
interrupt entry	≠ OS service	ISA 最小入口 vs 软件处理与唤醒	硬件越界拥有 OS
peak bandwidth	≠ effective bandwidth	理想 beat 速率 vs 完整事务有效数据率	漏协议开销
MMIO address	≠ memory data	设备寄存器契约 vs 普通可缓存对象	错用缓存/重排

11 做题控制协议

1. 标 initiator、target、数据方向、共享资源与 completion 条件；
2. 把 request、arbitration、address/control、data beat、response 分开；
3. 逐阶段写资源 Owner 和允许重叠的 CPU/设备工作；
4. 带宽先算完整事务有效数据/总时间，再与峰值比较；

5. I/O 方式题回答谁等待、谁搬数据、谁通知；
6. 中断题分 pending、accept、entry、software service、return；
7. DMA 题分 setup、transfer、completion，并单列一致性和 OS handoff。

压缩成第一动作

看到总线或 I/O 题, 先问: 当前谁拥有控制权? 当前事件只是 request, 还是已经形成 transaction 或 completion evidence ?

12 全科坐标与跨册交接

Map Coordinate: Resource / Interface

CO-08 负责共享线路、控制器状态、仲裁、事务和完成证据。Interrupt 解决“完成后怎样通知 CPU”, DMA 解决“数据怎样批量搬运”; 两者可组合但不能互相替代。X-B03/OS-05 接管 mapping、请求队列、completion 和 wakeup。

本册 Cost 要把事务有效数据、地址/控制/等待周期、仲裁、方向切换和可重叠阶段写全; 总线峰值、有效带宽、设备 latency 与 CPU 指令时间不是同一个量。若访问是 MMIO, 属性可能绕过普通 Cache, 但仍需回到 CO-07/B02 的权限与地址接口。

13 考纲映射与来源处理

考点	本册机制	接口
总线结构/仲裁/定时	角色、事务、握手、有效带宽	CO-03/05
I/O 接口与编址	寄存器状态机、MMIO/isolated I/O	CO-02
轮询/中断	等待、搬运、通知; 硬件精确入口	X-B01/X-B03
DMA	setup、仲裁、transfer、completion	OS-05/X-B03

一句话

总线题追踪“谁在何时拥有共享线路”, I/O 题追踪“谁推进状态、谁搬数据、什么证据证明完成”; 中断与 DMA 都只是跨层 handoff, 不替 OS 完成软件责任。

本册吸收旧《总线结构》《I/O 控制方式》《中断处理》《DMA》及相关控制器卡片; 空白《总线计算》《I/O 接口》不产生知识更新, OS 的阻塞、唤醒、buffer policy 与调度保留给各自 Owner。